Implementation of a Database for a Home Inventory Tracking System

by

Crt Alt Elite

1.0 Introduction and Background

Overview:

In modern households, there is often a battle with individuals attempting to maintain accurate and current records of their household possessions. This becomes a major problem when tracking valuables, managing household assets, or even making insurance claims, and can result in slow or rejected insurance claims after unforeseen incidents. The purpose of this Home Inventory Tracker System project is to design and implement a database system for this scenario, allowing individuals to accurately record, categorize and manage information about their possessions. The aim of this project is to produce or create a database system that is well-structured and normalized. All necessary information will be centralized, and data integrity will be ensured.

Business Scenario & Motivation

Individuals own a wide range of personal items, ranging from electronics to appliances and furniture and even jewelry. Over time, it becomes very difficult to remember what they own, where they are stored, and their worth. This causes complications when filing insurance claims or determining the total value of household assets. The Home Inventory tracker system makes this process much easier by allowing individuals to organize their information about their belongings in a simple and reliable way. Each user or individual can create inventories for different areas in their home, add details about each item, and even upload images for proof and verification. The motivation behind this project is to provide users with a useful tool that helps them maintain accurate records of their assets, lower the risk of data loss, and make insurance processes much faster.

Identified entities:

1. User
   - Represent users who own 1 or more inventories.
   - Includes user credentials
2. Inventory
   - Represent a collection of items associated with a user
3. Item
   - Represent users' physical items being tracked
   - Stores information about the item

4. Category
    - Represent classification groups, e.g. Furniture, Electronics, Jewelry, for the organization of items.
5. Tag
    - Represent descriptive labels for items for searching and reporting
6. Image (Weak entity)
    - Stores photographs linked to items.
    - Depending on Item for existence, therefore a weak entity.
7. Report (Weak entity)
    - Represent generated reports for inventories.
    - Dependent on Inventory for existence, therefore a weak entity.

This scenario forms the basis of database design. The identified entities above were modeled based on the relationships and dependencies mentioned in the business rules below.

2.0 Business Rules

Identified business rules:

1. Each user has a unique account
2. Users can own multiple inventories. Each inventory belongs to ONE user.
3. Each inventory can have many items. Each item belongs to one inventory.
4. An item can belong to many categories. A category can contain many items.
5. An item can have many tags, and each tag can be related to many items.
6. An item can have many images, but each image must be linked to one item.
7. Each inventory can have multiple reports. Each report belongs to one inventory only.

Description of Business Rules Extracted from the Scenario:

The business rules above were drawn from how people generally record and manage their household belongings. Each user represents a homeowner who needs their own secure account. Users can create multiple inventories to manage their possessions or group them by room or category, but every item belongs to only one inventory. Since the same type of item may be in different categories, the system supports many-to-many relationships between items, categories, and tags. Supporting data, i.e. images and reports, depend on the items or inventories they describe. Altogether, these rules reflect the everyday process of documenting, classifying and verifying personal belongings.

How Business Rules Translate into Database Relationships:

One-to-Many (1: N)

- User -> Inventory
- Inventory -> Item
- Item -> Image
- Inventory -> Report

Many-to-Many (M: N)

- Item -> Category
- Item -> Tag

How these rules guided the design of the database:

The rules identified shape the entire database structure. A user having many inventories, i.e. ownership, was implemented through one-to-many relationships, using foreign keys like userID and inventoryID. Many-to-many relationships between items, categories, and tags were handled by creating junction tables, i.e. ITEM_CATEGORY and ITEM_TAG.

Images and reports were modeled as weak entities, due to the link to their parent tables. To maintain referential integrity, they were linked to their parents with cascading deletes.

To enforce data accuracy and uniqueness, constraints like UNIQUE, and NOT NULL, as well as appropriate data types, were implemented.

This produces a normalized database that decreases redundancies. It keeps data reliable and consistent. Additionally, it mirrors real-world asset management.

3.0 Entity–Relationship Design

Strong Entities

USER

Represents system users who create and manage inventory collections.

Attribute	Data Type	Constraints	Justification
userId	INTEGER	PRIMARY KEY, AUTO_INCREMENT	Unique identifier for each user
username	VARCHAR(50)	UNIQUE, NOT NULL	User's login name; must be unique
email	VARCHAR(100)	UNIQUE, NOT NULL	Contact email; used for authentication
passwordHash	VARCHAR(255)	NOT NULL	Encrypted password for security

createdDate	DATE	NOT NULL, DEFAULT CURRENT_DATE	Account creation timestamp
lastLogin	DATETIME	NULL	Tracks user activity patterns

Design Rationale: User is a strong entity as it exists independently and serves as the root of the ownership hierarchy. Passwords are stored as a hash for security compliance.

INVENTORY

Represents a collection of items belonging to a user

Attribute	Data Type	Constraints	Justification
inventoryId	INTEGER	PRIMARY KEY, AUTO_INCREMENT	Unique identifier for each inventory
userId	INTEGER	FOREIGN KEY → USER(userId), NOT NULL	Links inventory to owner
name	VARCHAR(100)	NOT NULL	Descriptive name for the collection
description	TEXT	NULL	Optional detailed description
createdDate	DATE	NOT NULL, DEFAULT CURRENT_DATE	Tracking creation date
lastUpdated	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Tracks last modification

Design Rationale: Inventory allows users to organize items into logical groups. The 1:N relationship with User reflects that each user can have multiple inventories, but each inventory belongs to one user.

ITEM

Represents individual physical items being tracked.

Attribute	Data Type	Constraints	Justification
itemId	INTEGER	PRIMARY KEY, AUTO_INCREMENT	Unique identifier for each item
inventoryId	INTEGER	FOREIGN KEY → INVENTORY(inventoryId), NOT NULL	Links item to its inventory
name	VARCHAR(100)	NOT NULL	Item name/title
description	TEXT	NULL	Detailed item description

purchasePrice	DECIMAL(10,2)	NULL	Original purchase cost
currentValue	DECIMAL(10,2)	NULL	Current estimated value
purchaseDate	DATE	NULL	When item was acquired
serialNumber	VARCHAR(100)	NULL	Manufacturer's serial number
condition	VARCHAR(50)	NULL	Current condition (e.g., 'Excellent', 'Good')
location	VARCHAR(100)	NULL	Physical storage location

Design Rationale: DECIMAL(10,2) for monetary values ensures precise financial calculations. Nullable fields accommodate varying levels of information availability. Serial numbers support warranty tracking and theft recovery.

CATEGORY

Represents classification groups for items

Attribute	Data Type	Constraints	Justification
categoryId	INTEGER	PRIMARY KEY, AUTO_INCREMENT	Unique identifier
name	VARCHAR(50)	UNIQUE, NOT NULL	Category name
description	TEXT	NULL	Category explanation
color	VARCHAR(7)	NULL	Hex color code for UI visualization

Design Rationale: Categories are independent entities that can be reused across all users. Color attribute supports visual organization in the user interface.

TAG

Represents flexible labels for items

Attribute	Data Type	Constraints	Justification
tagId	INTEGER	PRIMARY KEY, AUTO_INCREMENT	Unique identifier
name	VARCHAR(50)	UNIQUE, NOT NULL	Tag name

color	VARCHAR(7)	NULL	Hex color code for UI visualization

Design Rationale: Tags provide more flexible classification than categories, allowing multiple descriptive labels per item. They are system-wide entities for consistency.

Weak Entities

IMAGE

Stores images associated with items. Weak entity because images cannot exist without a parent item.

Attribute	Data Type	Constraints	Justification
imageId	INTEGER	PRIMARY KEY, AUTO_INCREMENT	Unique identifier within item
itemId	INTEGER	PRIMARY KEY, FOREIGN KEY → ITEM(itemId), NOT NULL	Part of composite PK; identifying relationship
fileName	VARCHAR(255)	NOT NULL	Original file name
filePath	VARCHAR(500)	NOT NULL	Storage location/URL
fileSize	BIGINT	NOT NULL	File size in bytes
uploadDate	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Upload timestamp

Design Rationale: Image is a weak entity because it depends on Item for its existence, uses a composite primary key, and has an identifying relationship with Item.

REPORT

Generated reports for inventories. Weak entity because reports are meaningless without their parent inventory.

Attribute	Data Type	Constraints	Justification
reportId	INTEGER	PRIMARY KEY, AUTO_INCREMENT	Unique identifier within inventory
inventoryId	INTEGER	PRIMARY KEY, FOREIGN KEY → INVENTORY(inventoryId), NOT NULL	Part of composite PK; identifying relationship

reportType	VARCHAR(50)	NOT NULL	Type of report (e.g., 'PDF', 'CSV')
generatedDate	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Creation timestamp
fileName	VARCHAR(255)	NOT NULL	Generated file name
totalValue	DECIMAL(12,2)	NULL	Aggregate inventory value

Design Rationale: Report is a weak entity because it exists solely in the context of an inventory, uses a composite primary key, and has an identifying relationship with Inventory.

Junction Tables (Many-to-Many Relationships)

ITEM_CATEGORY

Implements the many-to-many relationship between Item and Category.

Attribute	Data Type	Constraints	Justification
itemId	INTEGER	PRIMARY KEY, FOREIGN KEY → ITEM(itemId)	Composite PK component
categoryId	INTEGER	PRIMARY KEY, FOREIGN KEY → CATEGORY(categoryId)	Composite PK component
assignedDate	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Audit trail

Design Rationale: Items can belong to multiple categories, and categories can contain multiple items.

ITEM_TAG

Implements the many-to-many relationship between Item and Tag.

Attribute	Data Type	Constraints	Justification
itemId	INTEGER	PRIMARY KEY, FOREIGN KEY → ITEM(itemId)	Composite PK component
tagId	INTEGER	PRIMARY KEY, FOREIGN KEY → TAG(tagId)	Composite PK component
assignedDate	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Audit trail

Design Rationale: Items can have multiple tags, and tags can be applied to multiple items.

4.0 Relational Schema

Unnormalised Table Includes the Following Columns:

USER_ID (PK)	INVENTORY_ID (PK)	ITEM NAME	CONDITION	TAG_ID (PK)	FILE_PATH
USERNAME	INV_NAME	ITEM DESCRIPTION	LOCATION	TAG_NAME	FILE_SIZE
EMAIL	INV_DESCRIPTION	PURCHASE_PRICE	CATEGORY_ID (PK)	TAG_COLOUR	UPLOAD_DATE
PASSWORD_HASH	CREATED_DATE	CURRENT_VALUE	CAT_NAME	ASSIGNED_DATE	REPORT_ID (PK)
CREATED_DATE	LAST_UPDATED	PURCHASE_DATE	CAT_DESCRIPTION	IMAGE_ID (PK)	GENERATED_DATE
LAST_LOGIN	ITEM_ID (PK)	SERIAL_NO	CAT_COLOUR	FILE_NAME	FILE_NAME

Snippect of Unnormalised Structure:

Item_ID	Item_Name	Price	Category	Tag	Inventory_ID	Inventory_name	User_Name	User_ID	Purchase Date
1	Dell Laptop	1200	Electronics	Work, Office	1	Office Inventory	sjones	001	2023-01-12
2	Canon EOS 90D	1500	Cameras	Photography, Lens	2	Photography Gear	tlee	002	2022-07-05

Issues:

- Multiple values stored in a single cell
- Repeated information about the inventory and user for each item.
- High risk of update, insertion, and deletion anomalies.

First Normal Form (1NF)

Item_ID	Item_Name	Price	Category	Tag	Inventory_ID	Inventory_name	User_Name	User_ID	Purchase Date
1	Dell Laptop	1200	Electronics	Work	1	Office Inventory	sjones	001	2023-01-12
1	Dell Laptop	1200	Electroni	Office	1	Office	sjones	001	2023-01-12

			cs			Inventory			
2	Canon EOS 90D	1500	Cameras	Photography	2	Photography Gear	tlee	002	2022-07-05
2	Canon EOS 90D	1500	Cameras	Lens	2	Photography Gear	tlee	002	2022-07-05

Improvement:

- Each cell contains atomic values

Second Normal Form (2NF)

Rules: Must be in 1NF, and all non-key attributes must be fully functionally dependent on the entire primary key (no partial dependencies). Therefore we created 9 separated tables, namely: USER, INVENTORY, ITEM, CATEGORY, TAG, ITEM_CATEGORY, ITEM_TAG, IMAGE, and REPORT.

Analysis of Composite Keys:

ITEM_CATEGORY Table:

- Primary Key: (ITEM_ID, CATEGORY_ID)
- ASSIGNED_DATE depends on both ITEM_ID and CATEGORY_ID
- No partial dependencies

ITEM_TAG Table:

- Primary Key: (ITEM_ID, TAG_ID)
- ASSIGNED_DATE depends on both ITEM_ID and TAG_ID
- No partial dependencies

IMAGE Table Issue:

- Original: IMAGE_ID (PK), ITEM_ID (PK, FK)
- Problem: IMAGE_ID alone can identify a record, making ITEM_ID a partial dependency source
- FILE_NAME, FILE_PATH, FILE_SIZE, UPLOAD_DATE depend only on IMAGE_ID

REPORT Table Issue:

- Original: REPORT_ID (PK), INVENTORY_ID (PK, FK)
- Problem: REPORT_ID alone can identify a record

- REPORT_TYPE, GENERATED_DATE, FILE_NAME, TOTAL_VALUE depend only on REPORT_ID

Improvement:

- Each table now describes a single subject, and all non-key attributes depend entirely on the table's primary key.

Third Normal Form (3NF)

The schema is already in 3NF after the corrections made in 2NF. There were no transitive dependencies to remove. The database structure is now:

- Free from data redundancy
- Free from update anomalies
- Free from insertion anomalies
- Free from deletion anomalies
- Optimized for data integrity and efficient querying

5.0 SQL Queries

Simple Queries

Query 1 – List all users

Purpose: Displays all users registered in the system. Useful for checking the user base.

```
SELECT user_id, username, email
FROM User;
```

Explanation:

This query retrieves all rows from the User table showing their ID, username, and email. It demonstrates basic SELECT functionality.

Screenshot:

user_id	username	email
1	sjones	sjones@email.com
2	tlee	tlee@email.com
NULL	NULL	NULL

Query 2 – Show all items with their current value

Purpose: Lists all items in the system along with their current monetary value.

```
SELECT item_id, name, current_value
FROM Item;
```

Explanation:
This query shows basic information about items. It demonstrates simple SELECT statements with specific columns.
Screenshot:

item_id	name	current_value
1	Dell Laptop	800.00
2	Canon EOS 90D	1200.00
NULL	NULL	NULL

Intermediate Queries

Query 3 – List all items belonging to a specific inventory
Purpose: Retrieves all items stored in a specified inventory. Useful for inventory-specific reporting.

```
SELECT Inventory.name AS Inventory, Item.name AS Item
FROM Item
JOIN Inventory ON Item.inventory_id = Inventory.inventory_id
WHERE Inventory.name = 'Office Inventory';
```

Explanation:
This query uses an INNER JOIN to link Item and Inventory and filters by a specific inventory name. Demonstrates JOIN + WHERE clause.
Screenshot:

Inventory	Item
Office Inventory	Dell Laptop

Query 4 – Count how many items each user owns
Purpose: Calculates the total number of items each user possesses.

```
SELECT User.username, COUNT(Item.item_id) AS total_items
FROM User
JOIN Inventory ON User.user_id = Inventory.user_id
JOIN Item ON Inventory.inventory_id = Item.inventory_id
GROUP BY User.username;
```

Explanation:
This query aggregates data using COUNT() and GROUP BY, showing how to summarize item ownership per user.

Screenshot:

	username	total_items
▶	sjones	1
	tlee	1

Query 5 – Show most valuable items

Purpose: Displays items ordered by current value from highest to lowest.

```
SELECT name, current_value
FROM Item
ORDER BY current_value DESC;
```

Explanation:

This query demonstrates sorting using ORDER BY to rank items by monetary value.

Screenshot:

	name	current_value
▶	Canon EOS 90D	1200.00
	Dell Laptop	800.00

Complex Queries

Query 6 – List all items with their categories

Purpose: Shows the many-to-many relationship between items and categories.

```
SELECT Item.name AS Item, Category.name AS Category
FROM Item
JOIN Item_Category ON Item.item_id = Item_Category.item_id
JOIN Category ON Item_Category.category_id = Category.category_id;
```

Explanation:

This query demonstrates handling many-to-many relationships using a junction table Item_Category.

Screenshot:

	Item	Category
▶	Dell Laptop	Electronics
	Canon EOS 90D	Cameras

Query 7 – Show total inventory value per user

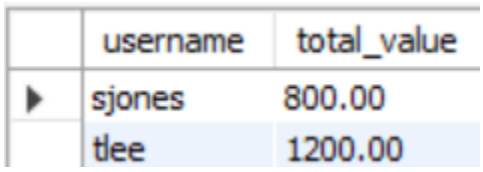
Purpose: Calculates the total value of items for each user. Useful for asset reporting.

```
SELECT User.username, SUM(Item.current_value) AS total_value
FROM User
JOIN Inventory ON User.user_id = Inventory.user_id
JOIN Item ON Inventory.inventory_id = Item.inventory_id
GROUP BY User.username;
```

Explanation:

Aggregates monetary values using SUM() and demonstrates multiple JOINs across three tables.

Screenshot:



username	total_value
sjones	800.00
tlee	1200.00

Query 8 – Find items with no tags assigned

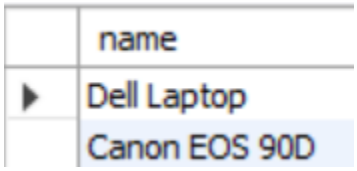
Purpose: Identifies items that do not have any tags assigned.

```
SELECT Item.name
FROM Item
LEFT JOIN Item_Tag ON Item.item_id = Item_Tag.item_id
WHERE Item_Tag.tag_id IS NULL;
```

Explanation:

Uses LEFT JOIN and NULL filtering to detect items without tags, showing how to handle missing relationships.

Screenshot:



name
Dell Laptop
Canon EOS 90D

Query 9 – Show images linked to items with inventory and user context

Purpose: Retrieves images of items along with the related inventory and user information.

```
SELECT User.username, Inventory.name AS Inventory, Item.name AS Item, Image.file_name
FROM Image
JOIN Item ON Image.item_id = Item.item_id
JOIN Inventory ON Item.inventory_id = Inventory.inventory_id
JOIN User ON Inventory.user_id = User.user_id;
```

Explanation:
Combines four tables to provide a full context of images in the system, demonstrating advanced JOIN usage.
Screenshot:

	username	Inventory	Item	file_name

Query 10 – Generate a report-like summary of inventories
Purpose: Provides a summary of total items and total value per inventory.

```
SELECT Inventory.name,
    (SELECT COUNT(*) FROM Item WHERE Item.inventory_id = Inventory.inventory_id) AS total_items,
            (SELECT SUM(current_value) FROM Item WHERE Item.inventory_id = Inventory.inventory_id) AS total_value
FROM Inventory;
```

Explanation:
Uses subqueries to generate a summary per inventory, simulating report generation and demonstrating aggregation with nested queries.
Screenshot:

	name	total_items	total_value
▶	Office Inventory	1	800.00
	Photography Gear	1	1200.00

6.0 Application Python Script

The python script creates a connection from the user to the database. It allows the user to query the database in a simple, understandable way.

The following code imports the MySQL library with the connector module (to establish a connection to the database) and defines functions for the first two queries:

```
import mysql.connector

# function for query to list all users
def listUsers(cursor):
    cursor.execute("SELECT user_id, username, email FROM User;")
    rows = cursor.fetchall()
    if not rows:
        print("\nNo users found\n")
    else:
        print("\nUsers:")
        for row in rows:
            print(f"User ID: {row[0]}, Username: {row[1]}, e-mail address: {row[2]}")
        print("\n")

# function for query to list all items and their values
def listItems(cursor):
    cursor.execute("SELECT item_id, name, current_value FROM Item;")
    rows = cursor.fetchall()
    if not rows:
        print("\nNo items found\n")
    else:
        print("\nItems:")
        for row in rows:
            print(f"Item ID: {row[0]}, Item name: {row[1]}, Item value: {row[2]}")
        print("\n")
```

The following code establishes the database connection and creates a menu for the user to choose which query to execute:

```
try:
    # establishing connection to database
    conn = mysql.connector.connect(
        host="localhost",
        user="marker",
        password="securepassword",
        database="inventory_system"
    )

    if conn.is_connected():
        print("Successfully connected to inventory_system database.")

    cursor = conn.cursor()

    # creating menu for user to choose quries and/or close the program
    while True:
        print("Query our database:")
        print("1.  List all users")
        print("2.  List all items")
        print("3.  List items in the office")
        print("4.  Count items for each user")
        print("5.  Order items by most to least valuable")
```

The following snippet shows how the code executes chosen queries:

```
155         choice = input("Enter your choice by entering only the number (e.g. to list all users just enter '1'): ")
156
157         if choice == "1":
158             listUsers(cursor)
159         elif choice == "2":
160             listItems(cursor)
161         elif choice == "3":
162             listItemsInOffice(cursor)
163         elif choice == "4":
164             countItemsOfUsers(cursor)
```

The following snippet shows how the user can exit the program, how errors are handled, and how the connection is kept clean:

```
177             elif choice == "11":
178                 print("Thanks for querying!")
179                 break
180             else:
181                 print("Input not valid. Try again.")
182
183     except mysql.connector.Error as err:
184         print(f"Connection failed: {err}")
185
186     finally:
187         # cleanup
188         if 'cursor' in locals():
189             cursor.close()
190         if 'conn' in locals() and conn.is_connected():
191             conn.close()
```

The script was tested by running the code and executing each query, which was successful. It was validated by comparing the output of the code to the output of the queries in MySQL.
The full script can be found at:
https://github.com/4121659/CrtAltElite_DB_Final/blob/main/sqlConnection.py

7.0 Testing and Validation

Testing Methodology

The database and SQL queries were tested to ensure correctness, reliability, and integrity of data. Testing was conducted in MySQL Workbench using the following steps:

Database Creation Validation

- Executed the CREATE DATABASE and CREATE TABLE scripts.
- Verified all tables were created with correct columns, data types, primary keys, and foreign keys.
- Checked for constraints such as UNIQUE and NOT NULL.

Data Insertion Testing

- Inserted sample data for User, Inventory, Item, Category, Item_Category, Tag, Item_Tag, Report, and Image tables.
- Verified that all data inserted correctly without violating constraints.

Query Testing

- Executed each SQL query (10 queries) individually.
- Validated outputs against expected results based on the sample data.
- Tested simple, intermediate, and complex queries for functionality.

Relationship Testing

- Verified one-to-many relationships: User -> Inventory, Inventory -> Item, Item -> Image, Inventory -> Report.
- Verified many-to-many relationships: Item -> Category, Item -> Tag using junction tables.

Data Integrity Checks

- Performed UPDATE and DELETE operations on parent tables to check cascading deletes on weak entities (Image and Report).
- Ensured that foreign key constraints prevented invalid data insertion.

Error Handling

- Monitored for SQL errors during query execution.
- Addressed issues such as duplicate entries, NULL violations, and invalid foreign key references.

Testing Results

Summary of test outcomes:

Test Category	**Outcome**
Database creation	Success – all tables created with correct schema
Data insertion	Success – all sample data inserted without constraint violations
Simple Queries	Success – returned all users and items as expected
Intermediate Queries	Success – joins and aggregations returned correct results

Complex Queries	Success – many-to-many joins and subqueries returned expected data
Referential integrity	Success – cascading deletes and foreign key constraints worked as intended

Example Outcomes:
Query 4 (Count of items per user) correctly showed:

	username	total_items
▶	sjones	1
	tlee	1

Query 7 (Total inventory value per user) correctly summed item values per user.

	username	total_value
▶	sjones	800.00
	tlee	1200.00

Error Handling and Limitations

Error Handling:
Constraints such as UNIQUE, NOT NULL, and foreign keys prevent invalid entries.
SQL exceptions were handled by reviewing error messages in MySQL Workbench and correcting input or query syntax.
Cascading deletes ensure weak entities like Image and Report are automatically removed when their parent entity is deleted.

Limitations:
Sample data is limited; real-world testing requires larger datasets.
The current implementation assumes manual query execution; integration with a Python frontend would require further testing.
Complex reporting queries may be slow for very large datasets without indexing or optimization.

Conclusion:

Testing demonstrated that the database and queries function correctly and maintain referential integrity. All SQL queries returned expected results, confirming the database design is robust and reliable.

8.0 Reflections and Discussion

The design of the database focused on normalization, referential integrity, and scalability, guaranteeing that data stayed accurate and consistent across every entity. Incorporating weak entities and many-to-many relationships necessitated thorough preparation to uphold correct constraints and cascading deletions. One key difficulty was handling intricate relationships and representing extensive normalized tables while maintaining clarity. Testing revealed minor problems related to data dependencies, which were addressed by modifying the schema. This project enhanced comprehension of relational design, normalization, and SQL querying while emphasizing the necessity of testing for reliability. This work was completed by Crt Alt Elite with reference to the prescribed textbook, MySQL and Python documentation for technical guidance.

.

9.0 Conclusion

The Home Inventory Tracking System successfully developed a practical, normalized database that efficiently oversees personal possessions. Testing verified that all queries functioned properly and that the database upheld data integrity. The project illustrated how effective database design can ease real-world asset tracking and enhance data accuracy.

The python script and .sql file can be found at: ***https://github.com/4121659/CrtAltElite_DB_Final***

Appendix

Team Members' Contribution

S/N	Name and Surname	Student Number	Key Contribution
1	Jean van Schalkwyk	4204301	SQL Querying Development & Testing and Validation
2	Kaamiel Isaacs	4129581	ERD Design & Explanation
3	Skye Jones (GL)	4122217	Database Implementation, Normalisation & Conclusion
4	Thaakirah Mosoval	4314422	Business Rules & Report Introduction
5	Uwais Cornelius	4121659	Python Integration & GitHub

References

[1] C. Coronel and S. Morris, Database Systems: Design, Implementation, & Management, 13th Edition.

[2] "geeksforgeeks," 23 July 2025. [Online]. Available: https://www.geeksforgeeks.org/dbms/database-design-in-dbms/. [Accessed 10 October 2025].

[3] S. Shaibu, "datacamp," 28 May 2024. [Online]. Available: https://www.datacamp.com/tutorial/normalization-in-sql. [Accessed 10 October 2025].

Plagiarism Declaration

We are aware and understand that plagiarism is wrong and punishable according to the UWC policy. We confirm that this work is our own and that we have not plagiarized our work nor allowed anyone to copy our report or code.

Database Implementation

Creating the Database

```
CREATE DATABASE inventory_system;

USE inventory_system;
```

```
-- USER TABLE
CREATE TABLE User (
    user_id INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(255) NOT NULL UNIQUE,
    email VARCHAR(255) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL
    );

-- INVENTORY TABLE
CREATE TABLE Inventory (
    inventory_id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    name VARCHAR(255) NOT NULL,
    description TEXT,
    created_date DATETIME DEFAULT CURRENT_TIMESTAMP,
    last_updated DATETIME,
    FOREIGN KEY (user_id) REFERENCES User(user_id)
);

-- ITEM TABLE
CREATE TABLE Item (
    item_id INT AUTO_INCREMENT PRIMARY KEY,
    inventory_id INT NOT NULL,
    name VARCHAR(255) NOT NULL,
    description TEXT,
    purchase_price DECIMAL(10, 2),
```

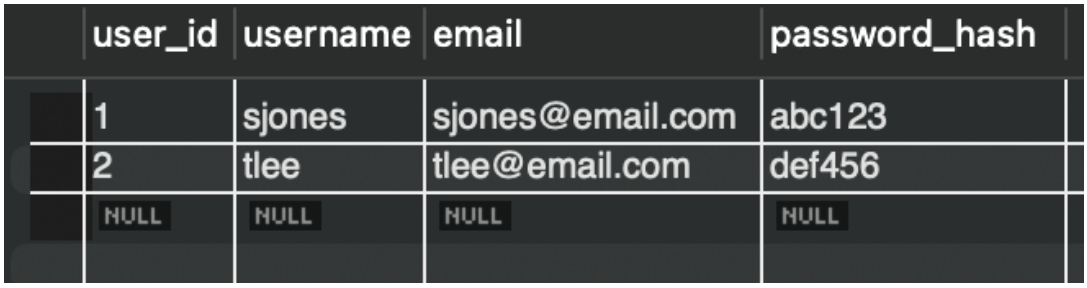
```
    current_value DECIMAL(10, 2),
    purchase_date DATE,
    serial_no VARCHAR(255),
    item_condition VARCHAR(255),
    location VARCHAR(255),
    FOREIGN KEY (inventory_id) REFERENCES Inventory(inventory_id)
);

-- CATEGORY TABLE
CREATE TABLE Category (
    category_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(255) NOT NULL UNIQUE,
    description TEXT,
    colour VARCHAR(255)
);

-- ITEM_CATEGORY
CREATE TABLE Item_Category (
    item_id INT NOT NULL,
    category_id INT NOT NULL,
    assigned_date DATE,
    PRIMARY KEY (item_id, category_id),
    FOREIGN KEY (item_id) REFERENCES Item(item_id),
    FOREIGN KEY (category_id) REFERENCES Category(category_id)
);
```

```
-- TAG TABLE
CREATE TABLE Tag (
    tag_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(255) NOT NULL UNIQUE,
    colour VARCHAR(255)
);

-- ITEM_TAG (JUNCTION TABLE)
CREATE TABLE Item_Tag (
    item_id INT NOT NULL,
    tag_id INT NOT NULL,
    assigned_date DATE,
    PRIMARY KEY (item_id, tag_id),
    FOREIGN KEY (item_id) REFERENCES Item(item_id),
    FOREIGN KEY (tag_id) REFERENCES Tag(tag_id)
);

-- REPORT TABLE
CREATE TABLE Report (
    report_id INT AUTO_INCREMENT PRIMARY KEY,
    inventory_id INT NOT NULL,
    report_type VARCHAR(255),
    generated_date DATETIME DEFAULT CURRENT_TIMESTAMP,
    file_name VARCHAR(255),
    total_value DECIMAL(12,2),
    FOREIGN KEY (inventory_id) REFERENCES Inventory(inventory_id)
```

```
);


-- IMAGE TABLE
CREATE TABLE Image (
    image_id INT AUTO_INCREMENT PRIMARY KEY,
    item_id INT NOT NULL,
    file_name VARCHAR(255),
    file_path VARCHAR(255),
    file_size INT,
    upload_date DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (item_id) REFERENCES Item(item_id)
);
```

Inserting Sample Data

```
-- USERS
INSERT INTO User (username, email, password_hash) VALUES
('sjones', 'sjones@email.com', 'abc123'),
('tlee', 'tlee@email.com', 'def456');
```



user_id	username	email	password_hash
1	sjones	sjones@email.com	abc123
2	tlee	tlee@email.com	def456
NULL	NULL	NULL	NULL

```
-- INVENTORY
INSERT INTO Inventory (user_id, name, description) VALUES
(1, 'Office Inventory', 'All office equipment'),
(2, 'Photography Gear', 'Cameras and lenses');
```

inventory_id	user_id	name	description	created_date	last_updat...
1	1	Office Inventory	All office equipment	2025-10-12 02:24:02	NULL
2	2	Photography Gear	Cameras and lenses	2025-10-12 02:24:02	NULL
NULL	NULL	NULL	NULL	NULL	NULL

-- ITEMS

INSERT INTO Item (inventory_id, name, description, purchase_price, current_value, purchase_date, serial_no, item_condition, location)

VALUES

(1, 'Dell Laptop', 'Work laptop', 1200, 800, '2023-01-12', 'DL12345', 'Good', 'Office'),

(2, 'Canon EOS 90D', 'Main camera', 1500, 1200, '2022-07-05', 'CN90D', 'Excellent', 'Studio');

item_id	inventory_id	name	description	purchase_pri...	current_val...	purchase_date	serial_no	item_conditi...	location
1	1	Dell Laptop	Work laptop	1200.00	800.00	2023-01-12	DL12345	Good	Office
2	2	Canon EOS 90D	Main camera	1500.00	1200.00	2022-07-05	CN90D	Excellent	Studio
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

-- CATEGORIES

INSERT INTO Category (name, description, colour)

VALUES ('Electronics', 'Electronic devices', 'Blue'),

('Cameras', 'Camera equipment', 'Black');

category_id	name	description	colour
1	Electronics	Electronic devices	Blue
2	Cameras	Camera equipment	Black
NULL	NULL	NULL	NULL

-- ITEM_CATEGORY

INSERT INTO Item_Category VALUES

(1, 1, '2023-03-01'),

(2, 2, '2023-04-15');

item_id	category_id	assigned_date
1	1	2023-03-01
2	2	2023-04-15
NULL	NULL	NULL